Connecting Amazon RDS to an EC2 Instance Using MySQL Client

To Begin with the Lab

Summary of the Lab

In this lab, an **Amazon RDS MySQL** database was deployed inside a VPC and connected securely to a Linux-based **Amazon EC2** instance acting as a MySQL client. Separate security groups were created for the EC2 instance and RDS database to follow a more realistic production-style architecture. Public access was disabled on the database, ensuring that connections could only originate from resources inside the VPC. After both resources were launched, the MySQL client was installed on the EC2 instance, the RDS endpoint was used to establish a connection, and sample database objects were created and verified. This demonstrated how application servers typically communicate with backend databases in real-world AWS environments.

Understanding Amazon RDS Private Database Access

Amazon RDS databases are typically deployed in private networks so they are not directly accessible from the internet. Instead, application servers or management servers running on **Amazon EC2** connect to the database using internal VPC networking. This improves security because database endpoints are hidden from public access while still allowing authorized applications to read and write data. Security Groups act as virtual firewalls, controlling which resources can communicate with the database.

Example:

An application server running on EC2 hosts an employee management portal. Employees submit information through the web application, and the EC2 server stores that data in an RDS MySQL database. Users never access the database directly; all communication flows through the application server, reducing security risks and exposing only the application interface to the internet.

Lab Steps

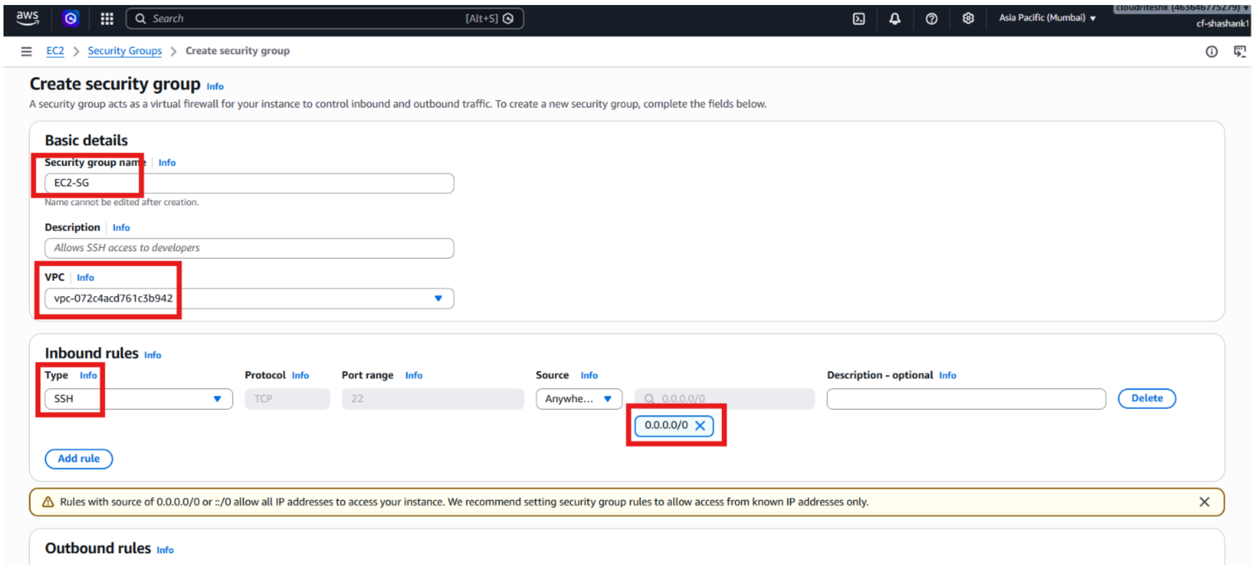
Step 1 – Create Security Group for EC2

- Sign in to the AWS Management Console.
- Navigate to **Amazon EC2 → Security Groups**.
- Click **Create Security Group**.
- Enter the security group name **EC2-SG**.

- Add a description for the EC2 MySQL client instance.
- Select the default VPC.
- Configure the inbound rule:

Type	Protocol	Port
SSH	TCP	22

- Leave outbound rules as default (**All Traffic**).



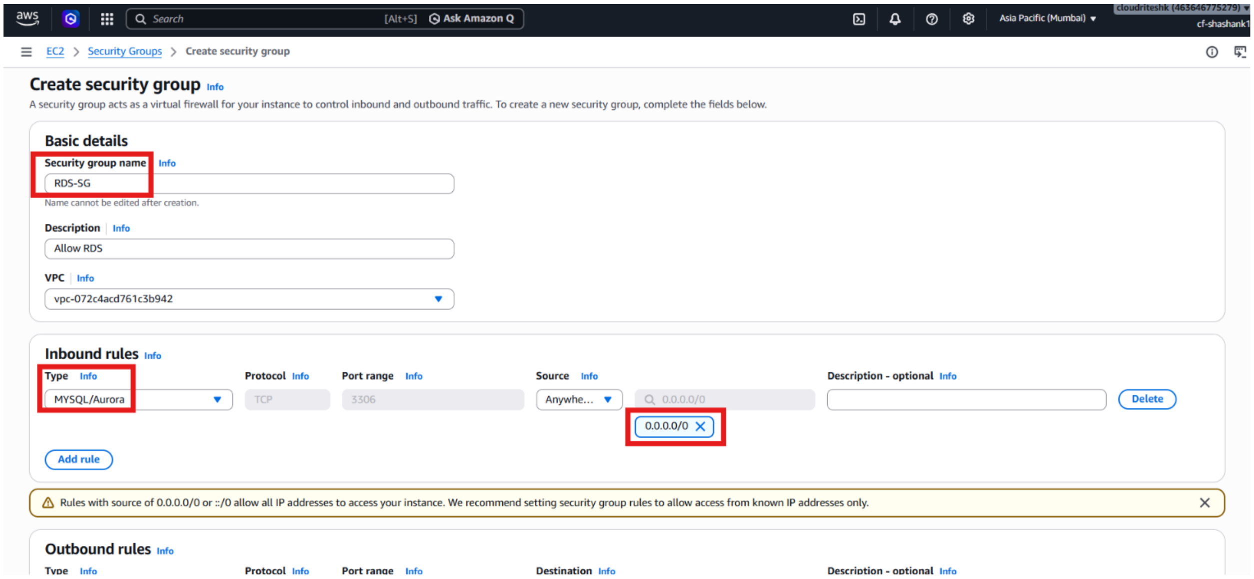
- Click **Create Security Group**.

Step 2 – Create Security Group for RDS

- Navigate to **Amazon EC2 → Security Groups**.
- Click **Create Security Group**.
- Enter the security group name **RDS-SG**.
- Add a description for the RDS database.
- Select the same VPC used by the EC2 instance.
- Configure the inbound rule:

Type	Protocol	Port
MySQL/Aurora	TCP	3306

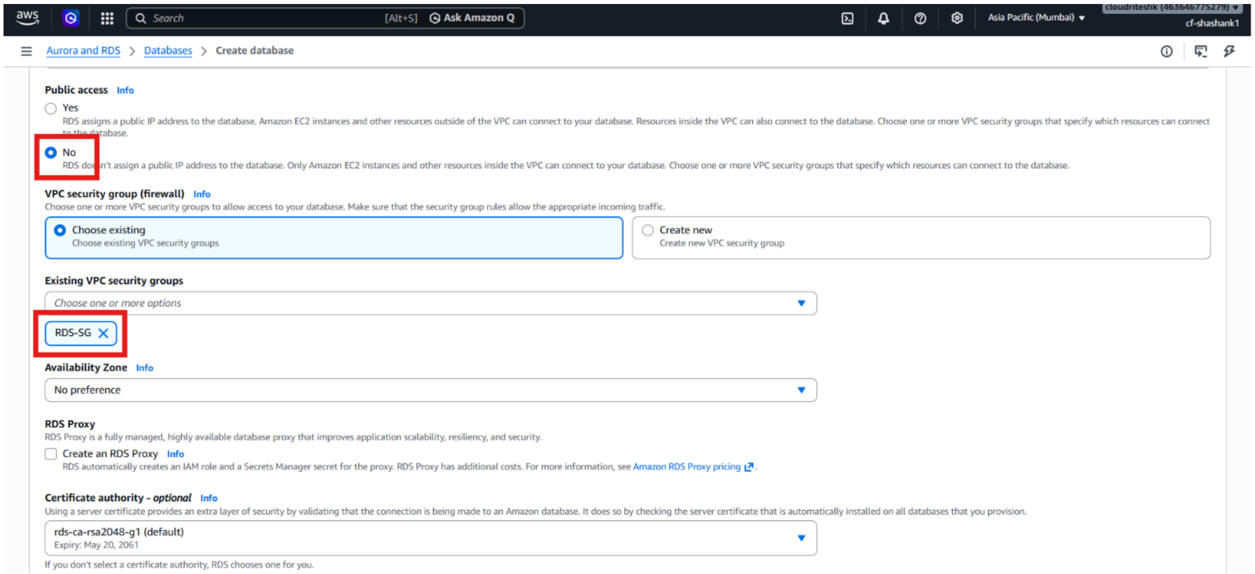
- Leave outbound rules as default (**All Traffic**).



- Click **Create Security Group**.

Step 3 – Create the Amazon RDS Database

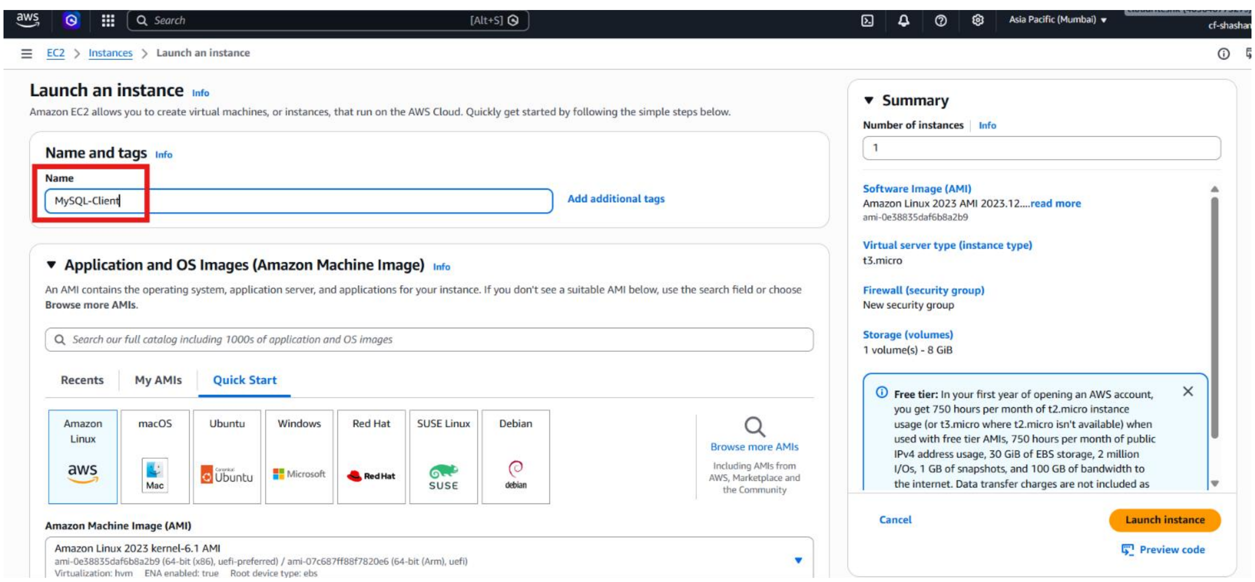
- Navigate to **Amazon RDS** → **Databases**.
- Click **Create Database**.
- Select **Standard Create**.
- Choose **MySQL**.
- Select the default MySQL 8.x version.
- Choose **Free Tier**.
- Enter a database identifier such as **myappdb**.
- Configure master credentials:
  - Username: **admin**
  - Password: Your chosen password
- Keep the default instance configuration.
- Set storage to 20 GB.
- Select the default VPC.
- Disable **Public Access**.
- Under **VPC Security Groups**, remove the default security group.
- Attach **RDS-SG**.



- Leave the remaining settings as default.
- Click **Create Database**.
- Wait until the database status changes to **Available**.

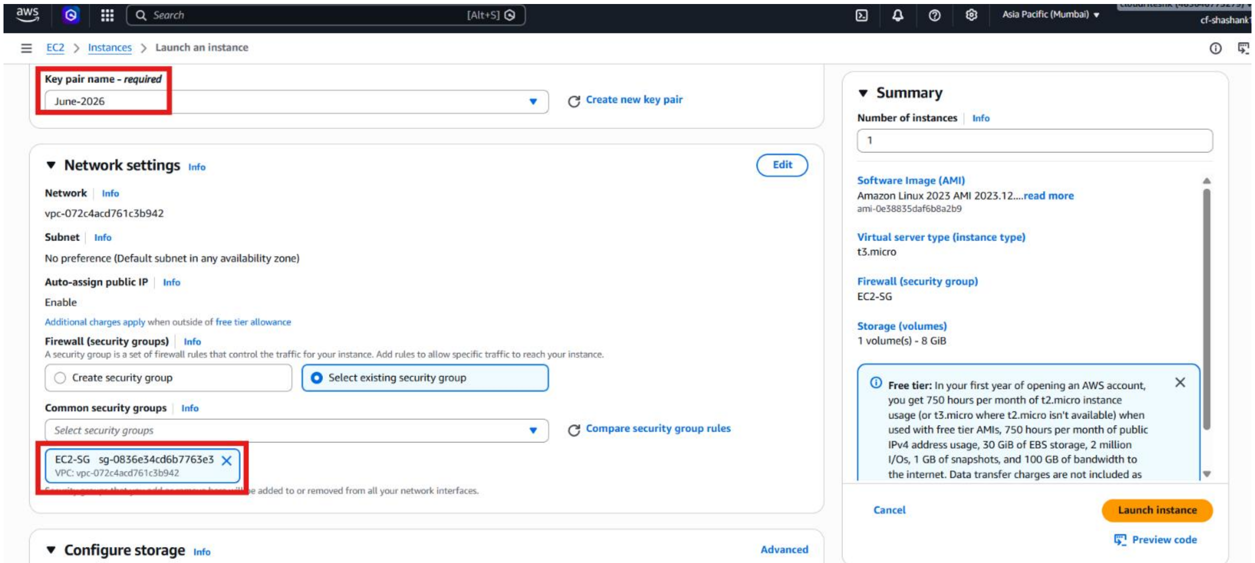
Step 4 – Launch the EC2 MySQL Client Instance

- Navigate to **Amazon EC2 → Instances**.
- Click **Launch Instance**.
- Enter the name **MySQL-Client**.
- Select **Amazon Linux 2023**.



- Choose **t3.micro**.
- Select an existing key pair.
- Ensure the instance is launched in the same VPC as the RDS instance.
- Enable a public IP address.

- Under **Network Settings**, select **Existing Security Group**.
- Attach **EC2-SG**.
- Click **Launch Instance**.

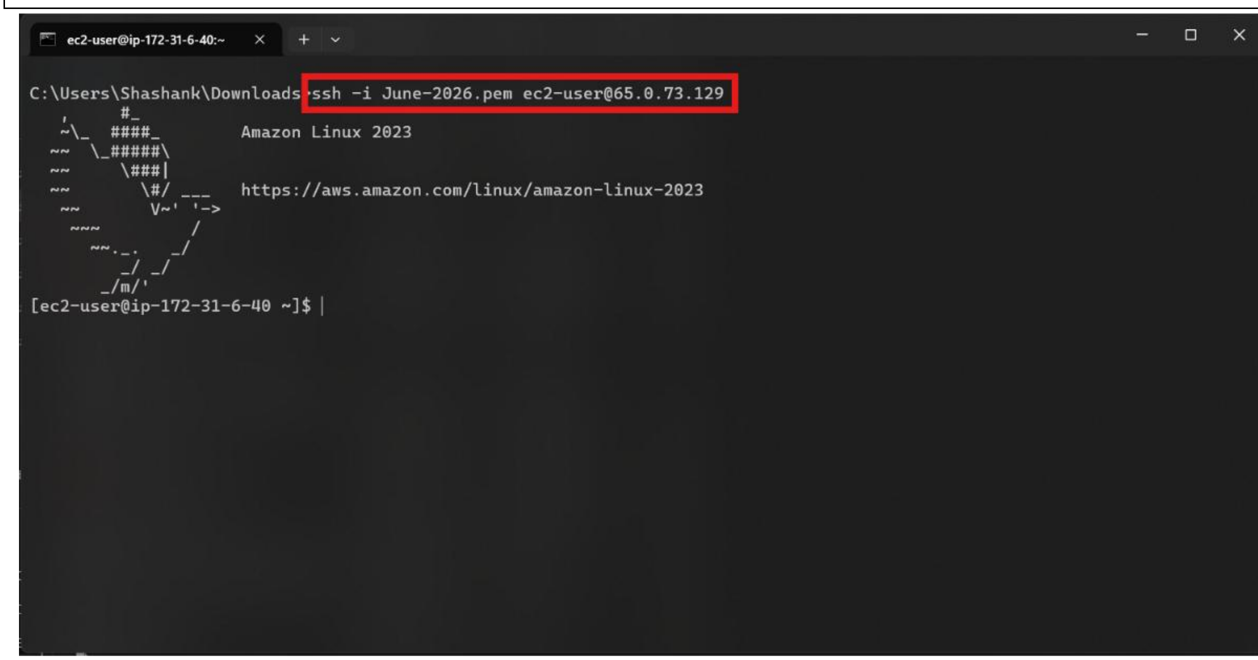


- Wait until the instance enters the **Running** state.

Step 5 – Connect EC2 to RDS

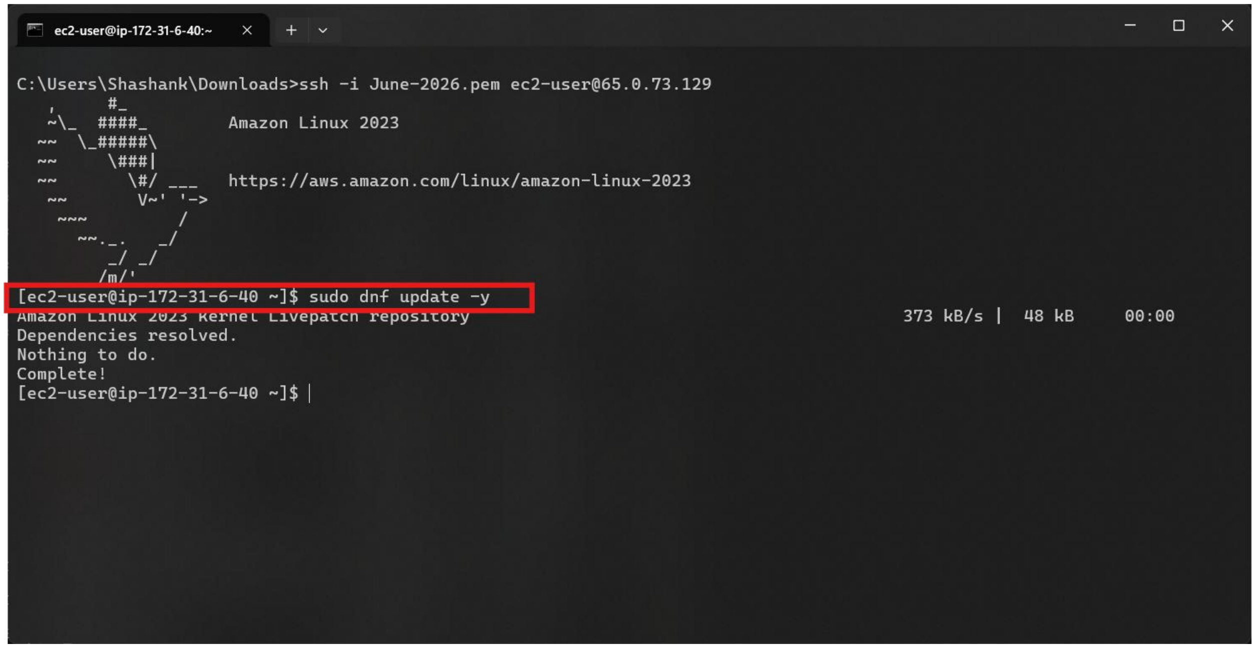
- Connect to the EC2 instance using SSH.

```
ssh -i your-key.pem ec2-user@EC2-Public-IP
```



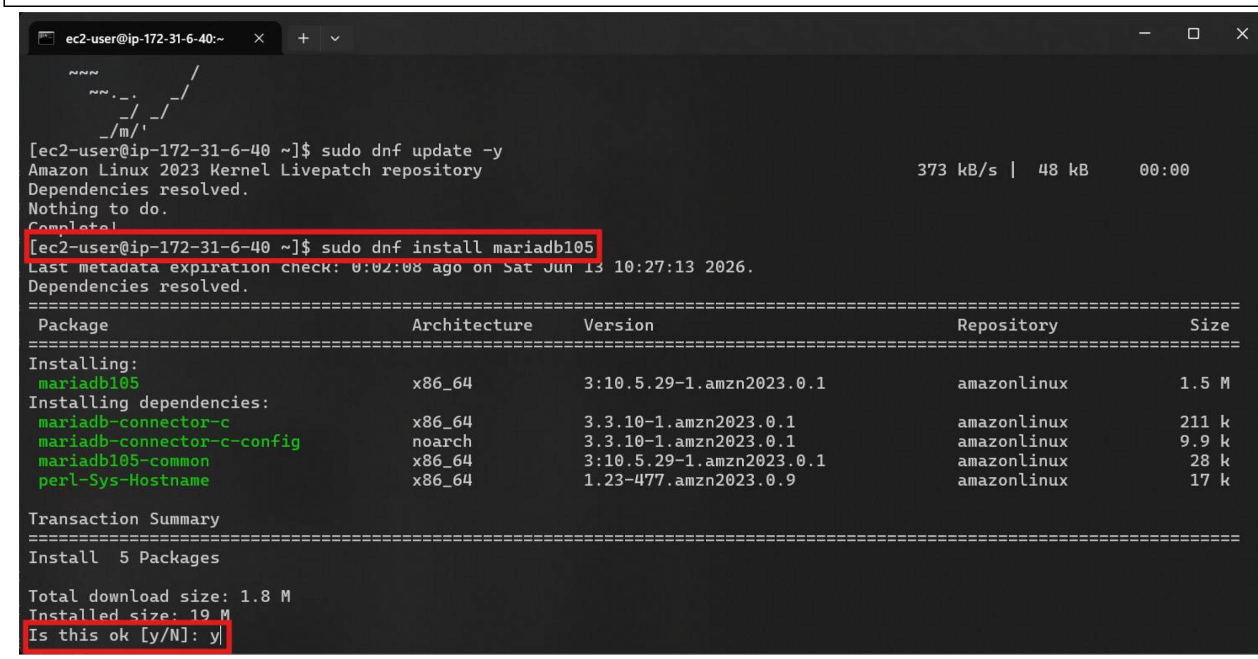
- Update the package repository.

```
sudo dnf update -y
```



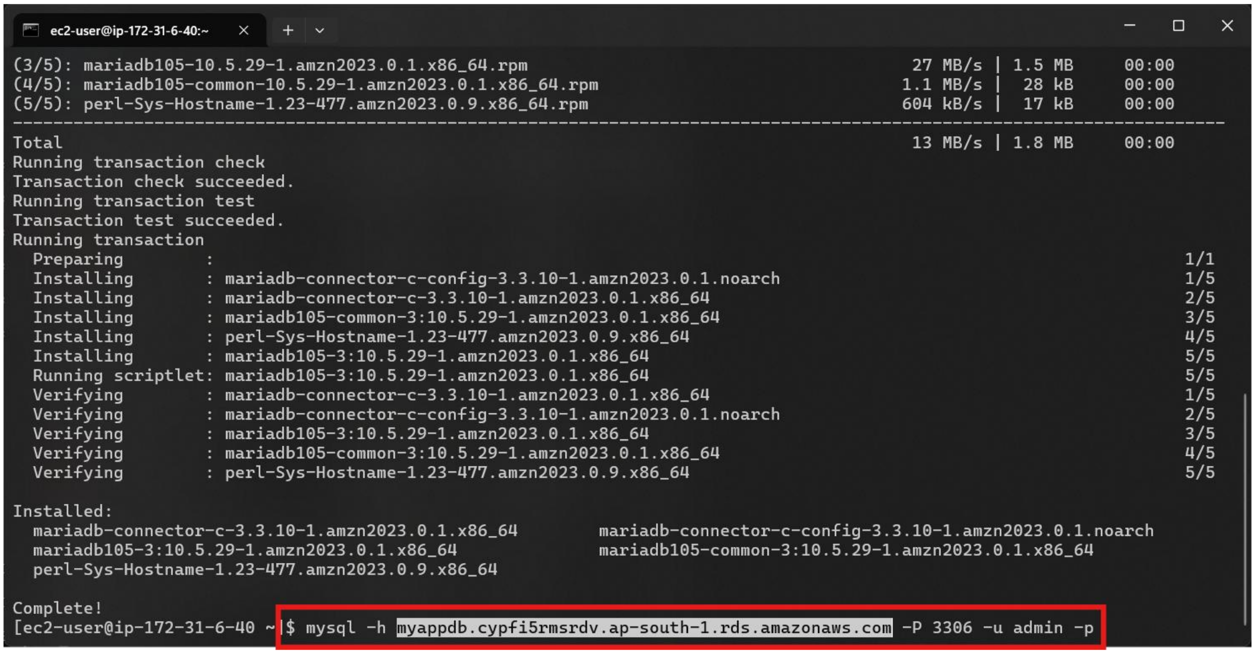
- Install the MySQL-compatible client.

```
sudo dnf install mariadb105
```

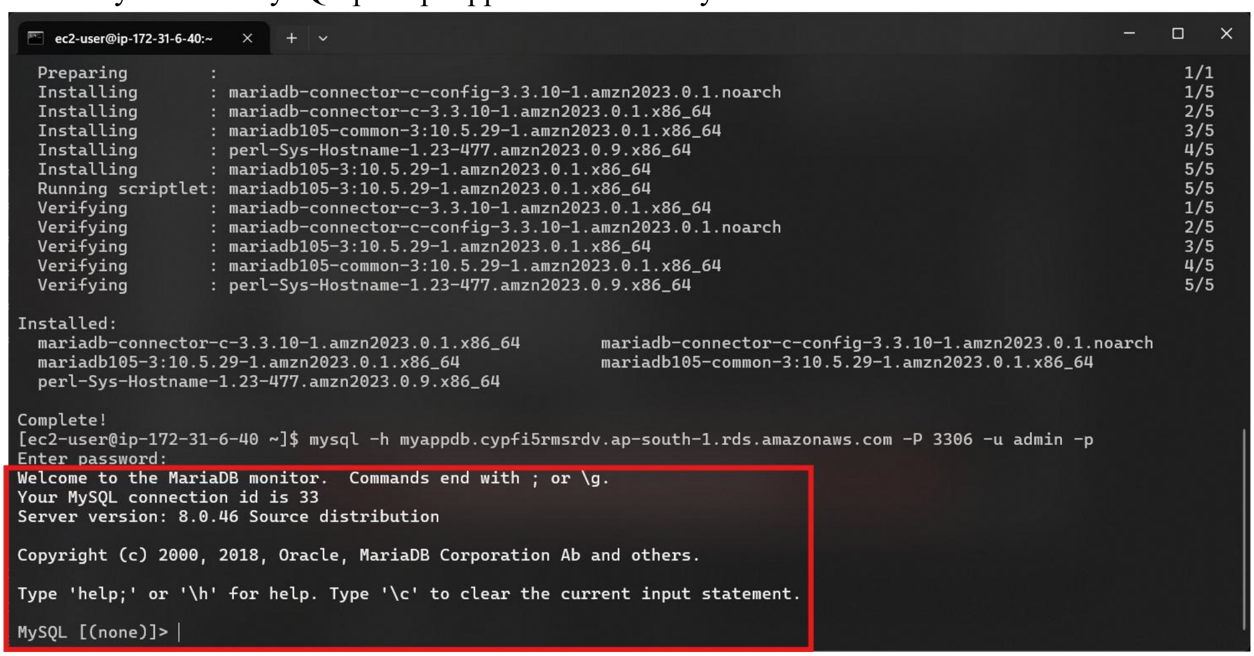


- Copy the RDS endpoint from the RDS console.
- Connect to the database.

```
mysql -h RDS-ENDPOINT -P 3306 -u admin -p
```



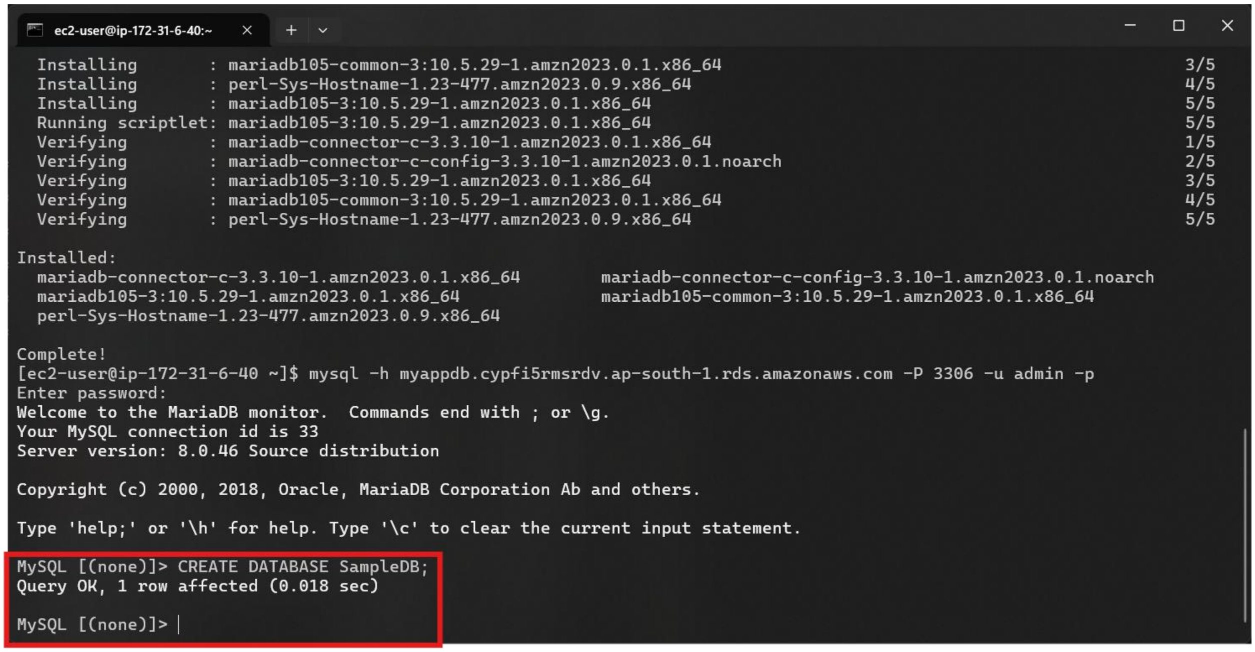
- Enter the database password when prompted.
- Verify that the MySQL prompt appears successfully.



Step 6 – Create and Manage Database Objects

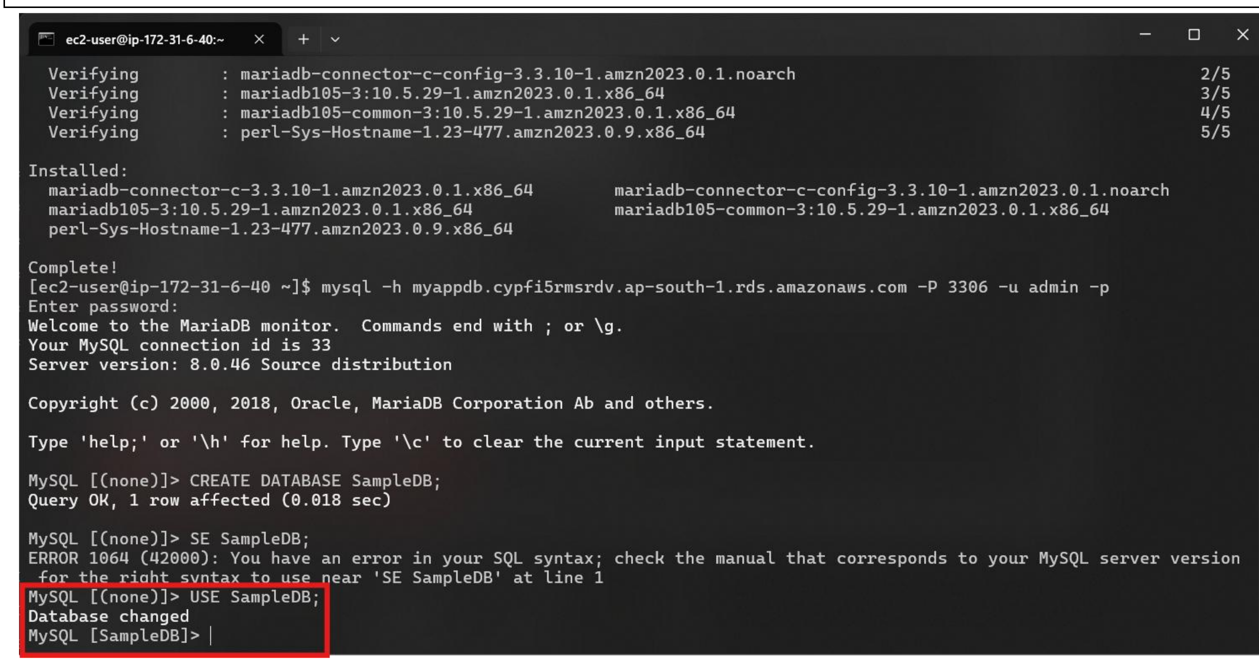
- Create a sample database.

```
CREATE DATABASE SampleDB;
```



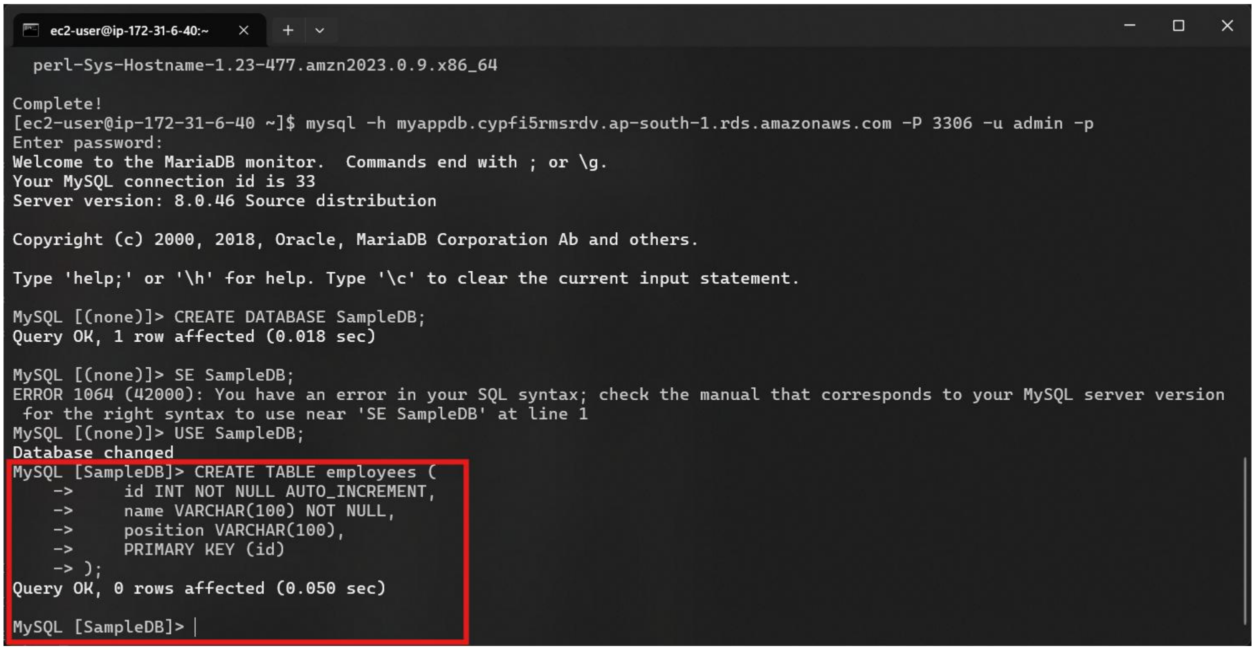
- Switch to the database.

```
USE SampleDB;
```



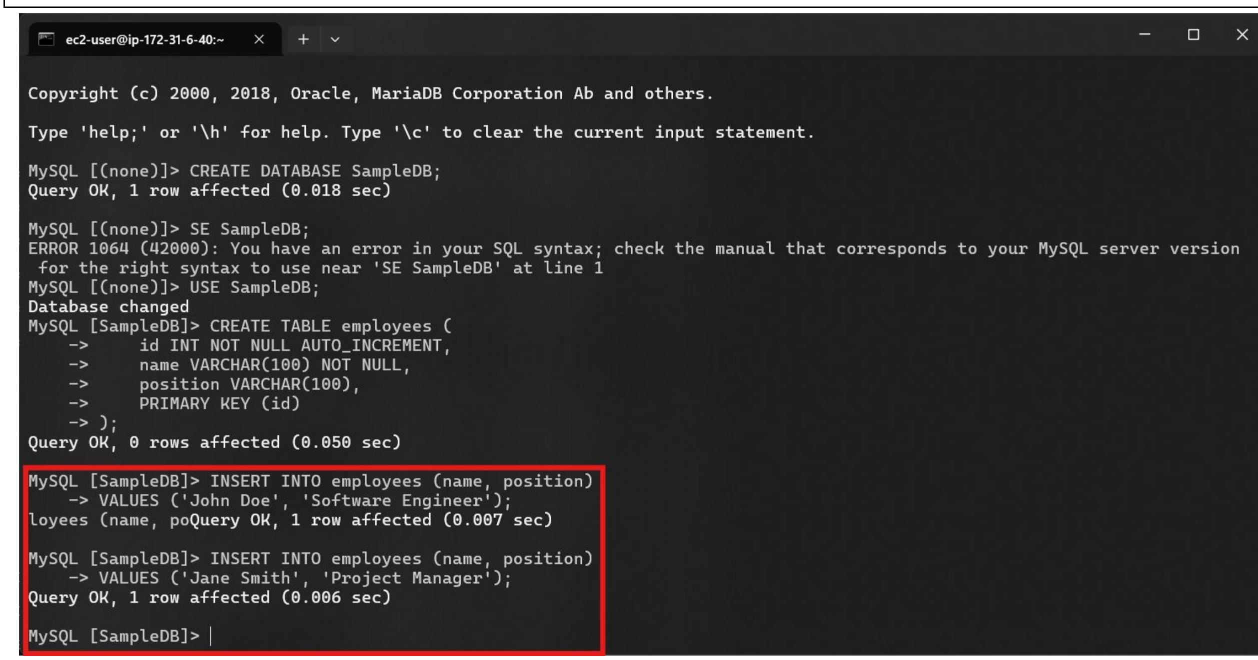
- Create a sample table.

```
CREATE TABLE employees (
   id INT NOT NULL AUTO_INCREMENT,
   name VARCHAR(100) NOT NULL,
   position VARCHAR(100),
   PRIMARY KEY (id)
);
```



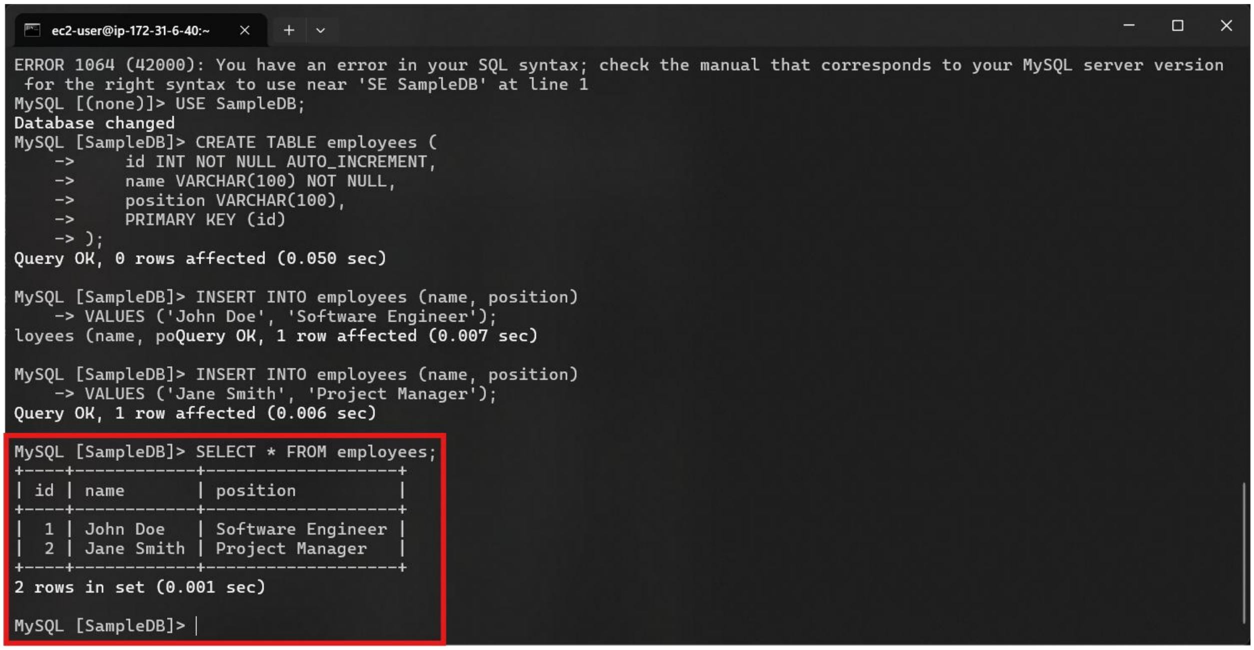
- Insert sample records.

```
INSERT INTO employees (name, position)
VALUES ('John Doe', 'Software Engineer');
INSERT INTO employees (name, position)
VALUES ('Jane Smith', 'Project Manager');
```



- Verify the inserted records.

```
SELECT * FROM employees;
```



```
ec2-user@ip-172-31-6-40:~
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version
 for the right syntax to use near 'SE SampleDB' at line 1
MySQL [(none)]> USE SampleDB;
Database changed
MySQL [SampleDB]> CREATE TABLE employees (
    ->     id INT NOT NULL AUTO_INCREMENT,
    ->     name VARCHAR(100) NOT NULL,
    ->     position VARCHAR(100),
    ->     PRIMARY KEY (id)
    -> );
Query OK, 0 rows affected (0.050 sec)

MySQL [SampleDB]> INSERT INTO employees (name, position)
    -> VALUES ('John Doe', 'Software Engineer');
loyees (name, poQuery OK, 1 row affected (0.007 sec)

MySQL [SampleDB]> INSERT INTO employees (name, position)
    -> VALUES ('Jane Smith', 'Project Manager');
Query OK, 1 row affected (0.006 sec)

MySQL [SampleDB]> SELECT * FROM employees;
+----+------------+-------------------+
| id | name       | position          |
+----+------------+-------------------+
|  1 | John Doe   | Software Engineer |
|  2 | Jane Smith | Project Manager   |
+----+------------+-------------------+
2 rows in set (0.001 sec)

MySQL [SampleDB]> |
```

- Confirm that the records are displayed successfully.

Verification

- The **Amazon RDS MySQL** database was created successfully.
- The database remained private and inaccessible from the public internet.
- The **Amazon EC2** instance connected successfully using the MySQL client.
- A database named **SampleDB** was created.
- The **employees** table was created successfully.
- Sample employee records were inserted and retrieved.
- Communication between **Amazon EC2** and **Amazon RDS** was verified successfully.